

Smart Air Quality Monitor

Extended Technical Documentation

Phase 1: Baseline System • Phase 2: FreeRTOS Deep Sleep Implementation

Version 2.0 — May 2026

STM32F103C8T6 | ESP-01 (ESP8266) | ESP32-S3 | FreeRTOS v10.3 | ThingSpeak

Abstract

This document extends the baseline Smart Air Quality Monitor engineering report to cover the full development arc of the project: from initial super-loop firmware through the production FreeRTOS multi-task architecture, and into the next design phase—an RTOS-aware deep-sleep power management strategy. The baseline system integrates an STM32F103C8T6 sensing node, an ESP-01 wireless bridge, and an ESP32-S3 cloud gateway. The extended phase replaces the polling-delay loop with a semaphore-driven task model and adds a power-manager task that gates sensor rail voltages, persists calibration data in RTC backup registers, and enters STM32 Stop mode between measurement cycles to reduce average current consumption by approximately 70 %.

This document serves both as a technical reference for the implemented firmware and as a forward-looking design guide for engineers extending the platform.

PART ONE: BASELINE SYSTEM

1. System Architecture Overview

1.1 Design Philosophy

The baseline system follows a strict three-tier hierarchy. At the lowest tier, the STM32F103C8T6 acquires raw sensor data and formats it into a compact binary frame. The middle tier, an ESP-01 (ESP8266) module, validates the UART frame and re-transmits the payload wirelessly using the ESP-NOW protocol. At the top tier, an ESP32-S3 gateway receives the wireless packet, logs it with RSSI metadata, and periodically uploads readings to the ThingSpeak cloud platform.

Modularity is the central design principle: each processor has a single, well-defined responsibility. The STM32 never handles wireless networking; the ESP-01 never stores or aggregates data. This isolation simplifies debugging, enables independent firmware upgrades, and allows the sensor node to be battery-operated while the gateway remains mains-powered.

1.2 Hardware Platform at a Glance

Component	Role	Key Specification
STM32F103C8T6	Sensor node, OLED, UART TX	ARM Cortex-M3 @ 72 MHz, 64 KB flash, 20 KB SRAM
MQ135 module	Multi-gas air quality	SnO ₂ sensor, 5 V heater, ADC2 CH5 (PA5)
DHT22 (AM2302)	Temperature & humidity	Single-wire, ±0.5°C, ±2 %RH accuracy
SSD1306 OLED	Local display, 128×64	I2C1 @ 400 kHz, 1024-byte framebuffer
ESP-01 (ESP8266)	UART bridge, ESP-NOW TX	80 MHz, 96 KB SRAM, UART @ 115200 baud
ESP32-S3	Wireless gateway, cloud uplink	Dual-core LX7 @ 240 MHz, BLE 5.0, 802.11 b/g/n
ThingSpeak	Cloud dashboard & analytics	HTTP REST API, 15-second minimum update interval

1.3 Peripheral Pin Assignment

Peripheral	Signal	STM32 Pin	Mode
DHT22	DATA	PA1	Bidirectional I/O (TIM1-timed)
MQ135	ANALOG OUT	PA5	ADC2 CH5 — 239.5-cycle sample time
SSD1306	SCL / SDA	PB6 / PB7	I2C1 Alternate Function (AF)

Peripheral	Signal	STM32 Pin	Mode
ESP-01	TX / RX	PB10 / PB11	USART3 AF — 115200 baud, 8N1
Debug UART	TX / RX	PA9 / PA10	USART1 AF — printf retarget
5V rail switch	EN	PA2	GPIO output — controls MQ135 power
3.3V rail switch	EN	PA3	GPIO output — controls DHT22/OLED/ESP-01

2. Sensor Interface Design

2.1 DHT22 Temperature & Humidity Sensor

The DHT22 communicates over a single-wire bidirectional protocol that demands microsecond-level timing. The STM32 drives the data line low for at least 1 ms to issue a start signal, then releases the bus and waits for the sensor's 80 μ s acknowledgement pulse. Forty data bits follow, each encoded by pulse width: a 26–28 μ s high phase represents a logical zero, while a 70 μ s high phase represents a logical one. TIM1 is configured with a prescaler of 71, yielding exactly 1 μ s resolution at 72 MHz, making the timing requirement straightforward to satisfy.

The 40-bit frame carries 16 bits of humidity (big-endian, tenths of %RH), 16 bits of temperature (sign flag in bit 15, tenths of $^{\circ}$ C), and an 8-bit XOR checksum. Bit 15 of the temperature word signals sub-zero readings. Temperature compensation data from the DHT22 is passed directly to the MQ135 driver to improve gas concentration accuracy.

Driver API — dht22.h

```
void DHT22_Init(GPIO_TypeDef *port, uint16_t pin, TIM_HandleTypeDef *htim);
uint8_t DHT22_ReadData(DHT22_Data_t *data); // returns 1 on success
Minimum sampling interval: 2 seconds (enforced by the calling task)
```

Sign Extraction — Temperature Decoding

```
/* Temperature - bit 15 is the sign flag */
uint16_t raw_temp = ((uint16_t)(temp_h & 0x7F) << 8) | temp_l;
data->temperature = (float)raw_temp / 10.0f;
if (temp_h & 0x80)
    data->temperature = -data->temperature;
```

2.2 MQ135 Multi-Gas Semiconductor Sensor

The MQ135 uses a heated tin dioxide (SnO_2) layer whose electrical conductivity increases when reducing gases adsorb on its surface. The module implements a voltage-divider circuit between the variable sensor resistance R_s and a fixed load resistor R_L . The ADC measures the voltage across

RL, from which Rs is back-calculated. Gas concentration is then derived from the Rs / R0 ratio using power-law regression coefficients published in the MQ135 datasheet.

Electrical Model and Rs Calculation

. With the board's load resistor measured at 2.6 kΩ and the module powered from 5 V while the ADC reference is 3.3 V, both voltages must be declared correctly in the driver to avoid systematic error in every gas reading.

Power-Law Regression Coefficients

Gas	Coefficient a	Coefficient b	Typical Range
CO ₂ (Carbon Dioxide)	110.47	-2.862	400–5000 ppm
CO (Carbon Monoxide)	605.18	-3.937	1–50 ppm
NH ₃ (Ammonia)	102.20	-2.473	10–300 ppm
Alcohol (Ethanol)	77.255	-3.18	10–300 ppm
Toluene	44.947	-3.445	10–100 ppm
Acetone	34.668	-3.369	50–200 ppm

Temperature / Humidity Compensation

The MQ135 datasheet sensitivity curves are specified at 20°C and 33 %RH. A polynomial correction factor is applied to compensate for deviations from these reference conditions. The factor is clamped to the range [0.1, 3.0] to prevent wild corrections on sensor startup when the heater has not yet stabilised.

```
#define CORA 0.00035f
#define CORB 0.02718f
#define CORC 1.39538f
#define CORD 0.0018f

float MQ135_GetCorrectionFactor(float temperature, float humidity)
{
    float factor = CORA * temperature * temperature
                  - CORB * temperature
                  + CORC
                  - (humidity - 33.0f) * CORD;
    if (factor < 0.1f) factor = 0.1f;
    if (factor > 3.0f) factor = 3.0f;
    return factor;
}
```

R0 Calibration

R0 is the sensor resistance in clean air and is the single most important calibration parameter. It must be determined once in known-clean outdoor air after a 24-hour initial burn-in. Fifty ADC samples are averaged and divided by the clean-air Rs / R0 factor of 3.6 (from the datasheet) to derive R0 in kΩ.

```
float MQ135_CalibrateR0(uint16_t num_samples)
{
    float rs_sum = 0.0f;
    uint16_t valid = 0;
    for (uint16_t i = 0; i < num_samples; i++) {
        uint16_t adc = MQ135_SingleConversion();
        if (adc == 0) continue;
        float rs = MQ135_GetRs(adc);
        if (rs > 0.0f) { rs_sum += rs; valid++; }
        HAL_Delay(50);
    }
    if (valid == 0) return -1.0f;
    /* In clean air: Rs / R0 ≈ MQ135_CLEAN_AIR_FACTOR (3.6) */
    mq_r0 = (rs_sum / (float)valid) / MQ135_CLEAN_AIR_FACTOR;
    mq_calibrated = 1;
    return mq_r0;
}
```

2.3 SSD1306 OLED Display

The SSD1306 controller drives a 128×64 monochrome OLED over I2C1 at 400 kHz (Fast Mode). All pixel operations are performed on a 1024-byte framebuffer in SRAM; the complete buffer is then pushed to the controller with a single

after each frame. The display is re-initialised on every power cycle since its VDD is gated by the 3.3 V power switch (PA3) managed by the power-manager task.

3. Communication Protocol Stack

3.1 Binary Telemetry Frame

The STM32 serialises all sensor readings into a 34-byte binary frame transmitted over USART3 at 115200 baud. The structure is designed for minimal overhead while providing sufficient integrity guarantees for an indoor sensor link.

Byte(s)	Field	Value / Description
0	SOF (Start of Frame)	0xAA — fixed sentinel
1	LEN	30 — size of SensorPayload_t in bytes
2–31	SensorPayload_t	7 × float (28 B) + 1 × uint16_t (2 B)
32	CRC	XOR of bytes 0–31
33	EOF (End of Frame)	0x55 — fixed sentinel

Transmission time at 115200 baud is approximately 2.95 ms per frame, yielding a channel utilisation of just 0.15 % against the 2-second sampling interval. The XOR checksum, while not as robust as CRC-16, is sufficient for a low-noise short-distance UART link: bit-error rates on a 10 cm PCB trace are negligible, and the SOF/EOF delimiters provide synchronisation recovery if any byte is lost.

3.2 ESP-01 Bridge: State Machine Frame Parser

The ESP-01 firmware implements a byte-by-byte state machine rather than a fixed-interval read. This ensures that the bridge can resynchronise to the data stream automatically if the STM32 resets mid-transmission or if a frame is corrupted.

```
void loop() {
    while (Serial.available()) {
        uint8_t b = Serial.read();
        if (!in_frame) {
            if (b == FRAME_SOF) {          // 0xAA
                in_frame = true;
                rx_idx = 0;
                rx_buf[rx_idx++] = b;
            }
        } else {
            if (rx_idx < FRAME_LEN) rx_buf[rx_idx++] = b;
            if (rx_idx == FRAME_LEN) {
                in_frame = false;
                SensorPayload_t payload;
                if (validateFrame(rx_buf, &payload))
                    sendViaEspNow(&payload); // forward on success
                rx_idx = 0;
            }
        }
    }
}
```

3.3 ESP-NOW Wireless Link

ESP-NOW is a connectionless Espressif protocol operating in the 2.4 GHz ISM band. Unlike WiFi association, it requires no access point and achieves latency below 10 ms. The 30-byte payload is well within the 250-byte packet limit. The ESP-01 registers the ESP32-S3's MAC address as a peer during setup; no re-registration is required between transmissions.

Parameter	Value
Maximum payload	250 bytes
Typical latency	< 10 ms
Line-of-sight range	Up to 220 m
Maximum registered peers	20
Encryption support	Optional AES-128
Operating channel	1 (2.412 GHz), locked to avoid WiFi interference

3.4 ESP32-S3 Gateway: Hybrid WiFi / ESP-NOW

The gateway maintains ESP-NOW reception as its default state and activates the WiFi radio only during cloud upload windows (minimum 20-second interval enforced by ThingSpeak). After each upload, the gateway explicitly calls

are called to restore the ESP-NOW listener since WiFi association changes channel state. This two-phase radio management avoids packet loss during upload windows.

PART TWO: FREERTOS & DEEP SLEEP IMPLEMENTATION

4. The Case for FreeRTOS and Why HAL_Delay Falls Short

The original super-loop firmware calls

between measurement cycles. This approach is simple and reliable for a single-task application, but it has two critical drawbacks when the project grows in scope. First, the CPU is fully awake and spinning during the 2-second idle period—wasting approximately 45 mA that could otherwise be saved by entering a low-power mode. Second, there is no mechanism for concurrent activities: the display update and the UART transmission happen sequentially, and adding any new task (for example, a button debouncer or an alarm buzzer) requires inserting blocking calls into an already fragile timing chain.

FreeRTOS resolves both problems. The scheduler's preemptive priority system allows independent tasks to execute concurrently without one blocking the other. More importantly, the idle task hook and the ability to suspend the scheduler provide a clean integration point for hardware sleep modes, allowing the MCU to enter STM32 Stop mode when all tasks are waiting.

Design Decision: Why Stop Mode instead of Sleep Mode?

STM32F1 offers three low-power modes: Sleep, Stop, and Standby.

Sleep mode halts only the CPU core; all peripherals remain clocked, saving roughly 15 mA from the 45 mA active current.

Stop mode halts the CPU and all clocks except the LSI/LSE oscillator feeding the RTC. Current drops to approximately 30–60 μ A, a reduction of over 99% versus active mode.

Standby mode offers similar current to Stop but loses all SRAM contents, making it impractical here as the RTOS heap and task stacks must survive the sleep period.

Conclusion: Stop mode is the correct choice. It maximises power savings while preserving memory, and the RTC alarm provides a reliable wake-up source.

5. FreeRTOS Task Architecture

5.1 Task Inventory and Responsibilities

The production firmware partitions application logic into five FreeRTOS tasks. The key insight is that sensor acquisition and output tasks are structurally independent: there is no reason the display update must wait for the telemetry frame to be sent. Semaphores enforce the correct ordering while allowing the scheduler to interleave work efficiently.

Task Name	Priority	Stack (words)	Responsibility
PowerManagerTask (defaultTask)	High	384	Master orchestrator: gates rail voltages, sequences semaphores, arms RTC alarm, enters Stop mode
MQ135Task	Normal	384	Reads gas concentrations after 5V rail is stable; persists and restores R0 via RTC backup registers
DHT22Task	Normal	384	Reads temperature and humidity after 3.3V rail powers up; enforces the 2-second sensor stabilisation delay
DisplayTask	Below Normal	384	Re-initialises the OLED (fresh after power cycle) and renders all readings; lower priority ensures sensors run first
TelemetryTask	Normal	384	Snapshots shared data under mutex, prints to debug UART, and transmits the 34-byte binary frame to ESP-01

5.2 Synchronisation Primitives

Eight binary semaphores coordinate the task pipeline. Each semaphore is initialised at zero (by creating it with a count of 1 and immediately waiting on it). The PowerManagerTask releases semaphores to unblock downstream tasks in a defined sequence, then waits for completion semaphores before proceeding to the next step.

Semaphore	Released By	Consumed By	Meaning
sem_mq135_start	PowerManagerTask	MQ135Task	5V rail is stable; begin gas reading
sem_3v3_start	PowerManagerTask	DHT22Task	3.3V rail is on; begin DHT22 acquisition
sem_mq135_done	MQ135Task	PowerManagerTask	Gas readings are complete and stored
sem_dht_done	DHT22Task	PowerManagerTask	Temperature/humidity readings are complete
sem_display_start	PowerManagerTask	DisplayTask	All sensor data ready; render OLED
sem_tx_start	PowerManagerTask	TelemetryTask	All sensor data ready; transmit frame
sem_display_done	DisplayTask	PowerManagerTask	OLED update complete
sem_tx_done	TelemetryTask	PowerManagerTask	UART frame sent; safe to cut power

A single mutex,

structures from concurrent access. Each task reads sensor data into a local copy, then takes the mutex for the minimum time needed to copy the local data into the shared buffers. The TelemetryTask follows the same pattern in reverse: it acquires the mutex to take a consistent snapshot, releases the mutex, then spends as long as needed printing and transmitting without holding the lock.

5.3 Measurement Cycle Sequence

The following timeline describes one complete measurement cycle as orchestrated by the PowerManagerTask. The 20-second MQ135 warm-up window and the 2-second DHT22 stabilisation delay overlap: the MQ135 heater is energised first, and the 3.3 V rail (DHT22, OLED, ESP-01) is switched on 20 seconds later. Both sensors therefore finish at roughly the same time, minimising total active time before the node returns to Stop mode.

```
/* PowerManagerTask - one full cycle */

// 1. Enable 5V rail → start MQ135 warm-up
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, GPIO_PIN_SET);
osSemaphoreRelease(sem_mq135_start);

// 2. Wait 20 s for MQ135 heater to stabilise
osDelay(20000);

// 3. Enable 3.3V rail → DHT22 / OLED / ESP-01 power on
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);
osSemaphoreRelease(sem_3v3_start);

// 4. Wait for both sensor tasks to signal completion
osSemaphoreWait(sem_mq135_done, osWaitForever);
osSemaphoreWait(sem_dht_done,   osWaitForever);

// 5. Signal output tasks (display + telemetry) simultaneously
osSemaphoreRelease(sem_display_start);
osSemaphoreRelease(sem_tx_start);

// 6. Wait for both output tasks to finish
osSemaphoreWait(sem_display_done, osWaitForever);
osSemaphoreWait(sem_tx_done,      osWaitForever);

// 7. Cut all sensor power, arm RTC, enter Stop mode
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2 | GPIO_PIN_3, GPIO_PIN_RESET);
// → see Section 6 for Stop mode entry sequence
```

6. FreeRTOS Deep Sleep Implementation

6.1 The Challenge: RTOS and Hardware Sleep Modes

Entering a hardware sleep mode from within a FreeRTOS application requires care. The FreeRTOS SysTick interrupt drives the scheduler tick at 1000 Hz. If the CPU enters Stop mode while SysTick is running, the tick counter will be wrong when the CPU wakes up, causing all task delays and timeouts to be incorrect. Furthermore, if any task preempts the PowerManagerTask between the moment it arms the RTC alarm and the moment it calls the sleep instruction, the wake-up event could be missed.

The solution used in this firmware is to explicitly suspend the scheduler and disable the HAL tick source before entering Stop mode, then restore both immediately on wake. This is the safest approach on CMSIS-RTOS v1 without a dedicated low-power FreeRTOS port. All other tasks simply experience a longer apparent delay—equivalent to about 5 minutes—which is entirely acceptable for a 5-minute measurement cycle.

Why not use the FreeRTOS tickless idle?

FreeRTOS provides a `configUSE_TICKLESS_IDLE` option that suppresses the SysTick interrupt during idle periods, allowing the processor to enter Sleep mode automatically.

However, the STM32F1 port for tickless idle only supports Sleep mode (CPU halted, peripherals running), not Stop mode. Entering Stop mode from the tickless idle hook requires a custom port-level implementation that overrides `portSUPPRESS_TICKS_AND_SLEEP()`.

For a 5-minute cycle where active time is only ~25 seconds, the simpler approach of suspending the scheduler and sleeping from the `PowerManagerTask` is more transparent, easier to debug, and avoids tick compensation arithmetic. A tickless implementation would be preferable in a shorter-cycle design (e.g., <30 seconds).

6.2 RTC Alarm Configuration

The RTC on the STM32F103 is driven by the LSI (low-speed internal, ~40 kHz) oscillator, which continues running in Stop mode. The

. The RTC ALARM interrupt is routed through the EXTI17 line, which remains active in Stop mode and is capable of waking the CPU.

```
/* Arm RTC alarm for T + 5 minutes */
RTC_TimeTypeDef sTime = {0};
RTC_DateTypeDef sDate = {0};
HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN); // MUST read date to unlock shadow
regs

RTC_AlarmTypeDef sAlarm = {0};
sAlarm.AlarmTime.Hours = sTime.Hours;
sAlarm.AlarmTime.Minutes = sTime.Minutes + 5;
sAlarm.AlarmTime.Seconds = sTime.Seconds;
if (sAlarm.AlarmTime.Minutes >= 60) {
    sAlarm.AlarmTime.Minutes -= 60;
    sAlarm.AlarmTime.Hours = (sAlarm.AlarmTime.Hours + 1) % 24;
}
sAlarm.Alarm = RTC_ALARM_A;
HAL_RTC_SetAlarm_IT(&hrtc, &sAlarm, RTC_FORMAT_BIN);
```

① The shadow register unlock pattern (reading Date after Time) is required on the STM32F1 RTC. Omitting the `HAL_RTC_GetDate()` call causes `GetTime` to return stale values.

6.3 Entering and Exiting Stop Mode

The full Stop mode entry sequence suspends the RTOS scheduler, disables the HAL SysTick, enters Stop mode with the low-power voltage regulator enabled, and resumes normal operation on wake.

```
/* STEP 1: Suspend FreeRTOS scheduler — no task switch will occur */
vTaskSuspendAll();

/* STEP 2: Disable SysTick to prevent tick count corruption */
HAL_SuspendTick();

/* STEP 3: Enter Stop mode — CPU halts, only RTC keeps running */
/* PWR_LOWPOWERREGULATOR_ON reduces regulator quiescent current */
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

/* =====
 * <<<  RTC ALARM FIRES — CPU RESUMES HERE  >>>
 * ===== */

/* STEP 4: Restore PLL — Stop mode falls back to HSI (8 MHz) */
SystemClock_Config(); // re-locks PLL to 72 MHz

/* STEP 5: Re-enable SysTick and resume scheduler */
HAL_ResumeTick();
xTaskResumeAll();
```

① PWR_STOPENTRY_WFI (Wait For Interrupt) is preferred over WFE (Wait For Event) because WFI ensures the CPU wakes only on a genuine interrupt, preventing spurious wake-ups from pending event flags.

6.4 Clock Restoration After Wake

Stop mode disables the PLL and switches the system clock to the internal HSI oscillator (8 MHz). All peripheral baud rates and timer frequencies become incorrect until the PLL is re-locked. The solution is straightforward: calling

after wake runs the same clock initialisation sequence as at startup (HSE → PLL ×9 → 72 MHz), restoring all peripheral clocks transparently. The function takes approximately 1 ms to complete.

Critical Post-Wake Checklist

1. SystemClock_Config() — restore PLL before any peripheral access
2. HAL_ResumeTick() — re-enable SysTick so HAL_GetTick() counts correctly
3. xTaskResumeAll() — allow the scheduler to resume context switching
4. TIM1 does NOT need re-initialisation; its configuration register values survive Stop mode, but the counter register has advanced. DHT22_Init() does not reset the counter, so this is safe.
5. I2C1 may need a reset pulse if SCL/SDA lines were disturbed during power gating. SSD1306_Init() is called by DisplayTask on every cycle, handling this automatically.

7. R0 Persistence via RTC Backup Registers

The MQ135 calibration routine takes 50 ADC samples over 2.5 seconds and is designed to run in clean outdoor air. Running it on every wake from Stop mode would be incorrect (the air quality changes between measurements) and wasteful. The firmware uses two RTC backup registers to persist the calibrated R0 value across sleep cycles.

; if not (first boot), calibration is run and the result is written to both registers.

```
#define R0_BKP_MAGIC 0xA5A5u
#define R0_BKP_DR_VAL RTC_BKP_DR1 // stores R0 x10 as integer
#define R0_BKP_DR_MGC RTC_BKP_DR2 // magic number sentinel

if (HAL_RTCEx_BKUPRead(&hrtc, R0_BKP_DR_MGC) == R0_BKP_MAGIC) {
    /* Warm wake: restore persisted R0 */
    uint32_t r0_x10 = HAL_RTCEx_BKUPRead(&hrtc, R0_BKP_DR_VAL);
    MQ135_SetR0((float)r0_x10 / 10.0f);
} else {
    /* First boot: calibrate in clean air and save */
    float r0 = MQ135_CalibrateR0(MQ135_CAL_SAMPLES);
    HAL_RTCEx_BKUPWrite(&hrtc, R0_BKP_DR_VAL, (uint32_t)(r0 * 10.0f));
    HAL_RTCEx_BKUPWrite(&hrtc, R0_BKP_DR_MGC, R0_BKP_MAGIC);
}
```

① RTC backup registers are powered by the VBAT domain and survive both Stop mode and a main VDD power cycle (if a coin cell is connected to VBAT). On the STM32F103, 10 backup registers (DR1–DR10) are available, each storing 16 bits.

8. Power Budget Analysis

8.1 Baseline (Super-Loop) vs. FreeRTOS + Stop Mode

Component	Active (mA)	Stop / Idle (mA)	Baseline Avg (mA)	RTOS+Sleep Avg (mA)
STM32F103 @ 72 MHz	45	0.03 (Stop)	16.2	1.7
MQ135 Heater	150	0 (power-gated)	150.0	22.5
DHT22	1.5	0 (power-gated)	1.5	0.2
SSD1306 OLED	20	0 (power-gated)	10.0	1.5
ESP-01 (TX phase)	280	0 (power-gated)	15.4	2.3
Total	—	—	~193 mA	~28 mA

The MQ135 heater, which previously ran continuously at 150 mA, is now gated by the PA2 power switch and energised only during the 20-second warm-up window plus the brief measurement period (~25 seconds total per 5-minute cycle). This single change accounts for the majority of the ~85 % power reduction. Battery life on a 2000 mAh cell would extend from approximately 10 hours to over 70 hours.

8.2 Active Time Breakdown per 5-Minute Cycle

Phase	Duration	Notes
MQ135 heater warm-up	20 000 ms	5V rail on; MQ135 heating. DHT22/OLED/ESP-01 still off
3.3V rail power-on + DHT22 stabilisation	2 000 ms	Both rails now on; DHT22 waiting before first read
Sensor acquisition (MQ135 + DHT22 parallel)	~500 ms	50 ADC samples + 40-bit DHT22 frame
Display update (I2C DMA)	~25 ms	1024-byte framebuffer push at 400 kHz
UART telemetry frame TX	~3 ms	34 bytes @ 115200 baud
Stop mode (sleep)	~277 500 ms	All sensors gated; CPU at ~30 μ A
Total active per cycle	~22 528 ms	4.5% duty cycle on sensor rails

9. Stack Overflow Detection and Defensive Coding

The FreeRTOS configuration enables stack overflow checking at level 2 (), which paints the stack with a known pattern and verifies the pattern at each context switch. If overflow is detected, the hook function is called.

```
void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName)
{
    /* Transmit task name directly – printf may be unsafe here */
    HAL_UART_Transmit(&huart1, (uint8_t*)" [FATAL] Stack overflow: ", 24, 100);
    HAL_UART_Transmit(&huart1, (uint8_t*)pcTaskName, strlen(pcTaskName), 100);
    HAL_UART_Transmit(&huart1, (uint8_t*)"\r\n", 2, 100);
    __disable_irq();
    for (;;) { } // halt: do not attempt recovery
}
```

Each task is allocated 384 words (1536 bytes) of stack. The largest stack consumers are the DisplayTask (sprintf formatting, SSD1306 framebuffer writes) and TelemetryTask (local snapshot copies of both sensor structs). With the FreeRTOS minimum of 128 words and the application allocating 384, there is roughly 200 words of headroom per task—sufficient for nested HAL calls but worth monitoring with

during integration testing.

10. Known Issues, Bug Fixes, and Design Notes

Issue	Root Cause	Fix Applied
Struct field nh3 vs. nh4 mismatch	SensorPayload_t originally named the ammonia field nh3 while MQ135_GasReadings_t uses nh4	Field renamed to nh4 throughout; documented with comment
Race condition on shared sensor buffers	MQ135Task and DHT22Task wrote global dht_data / gas_data without synchronisation	mtx_sensor_data mutex created; all accesses use local copy + mutex-guarded swap
SysTick corruption across Stop mode	Entering Stop mode without suspending the scheduler corrupted the tick counter	vTaskSuspendAll() + HAL_SuspendTick() before HAL_PWR_EnterSTOPMode(), reversed on wake
RTC date shadow register not read	HAL_RTC_GetTime() returned stale values because GetDate() was not called	HAL_RTC_GetDate() call added immediately after GetTime() to unlock shadow registers
R0 recalibrated on every wake	No persistence mechanism; calibration ran in whatever air quality was present	RTC backup register sentinel (0xA5A5) added; calibration only on first boot
MQ135 ADC sampling time too short	Default ADC sample time was 1.5 cycles; MQ135 has high source impedance causing under-sampling	ADC_SAMPLETIME_239CYCLES_5 selected for maximum accuracy

11. FreeRTOS Configuration Reference

The following parameters in FreeRTOSConfig.h are directly relevant to the deep-sleep implementation and task management described in this document.

```

/* Kernel configuration */
#define configUSE_PREEMPTION          1      // preemptive scheduling
#define configTICK_RATE_HZ           1000   // 1 ms resolution
#define configMAX_PRIORITIES          7
#define configTOTAL_HEAP_SIZE         10240  // 10 KB FreeRTOS heap
#define configMINIMAL_STACK_SIZE      128    // words

/* Mutex and semaphore support */
#define configUSE_MUTEXES              1

/* Stack overflow detection (method 2: pattern checking) */
#define configCHECK_FOR_STACK_OVERFLOW 2

/* Newlib reentrant support (needed for printf / sprintf in tasks) */
#define configUSE_NEWLIB_REENTRANT    1

/* Port interrupt handlers mapped to CMSIS names */
#define vPortSVCHandler                SVC_Handler
#define xPortPendSVHandler              PendSV_Handler
#define xPortSysTickHandler             SysTick_Handler

```

① configUSE_NEWLIB_REENTRANT is essential because each FreeRTOS task needs its own reentrant newlib context for thread-safe sprintf and printf. Without this, concurrent format calls from different tasks would corrupt the heap.

12. Roadmap for Future Development

12.1 Immediate Enhancements

Several improvements are achievable without hardware changes. The MQ135 heater gating strategy currently runs the heater for exactly 20 seconds on every cycle. A more sophisticated approach would monitor the R_s / R_0 ratio and only proceed once the value has stabilised (a coefficient of variation below 2 % over ten consecutive samples, for example). This would self-adapt to ambient temperature and sensor age.

The RTC can also serve as a timestamping source for each measurement frame. Including a Unix-epoch timestamp in the telemetry payload would allow the ESP32-S3 gateway to detect dropped packets and report data gaps to ThingSpeak rather than forwarding stale readings.

12.2 Extending to Multiple Sensor Nodes

The ESP-NOW architecture supports up to 20 registered peers without modification. Adding additional STM32+ESP-01 sensor nodes requires only assigning each node a unique MAC filter entry on the ESP32-S3 gateway. The telemetry payload should be extended with a 1-byte node ID field to allow the gateway to demultiplex readings and post them to separate ThingSpeak channels.

12.3 Transitioning to Tickless Idle

For duty cycles shorter than 30 seconds, the overhead of scheduler suspension and clock restoration becomes a larger fraction of active time. In this regime, a proper FreeRTOS tickless idle implementation is preferable. This requires overriding

for sub-second wake intervals, enabling sub-second measurement cycles with negligible tick error.

12.4 Production Firmware Hardening

The current firmware halts on

. This enables fault analysis after a field failure without requiring JTAG access.

13. Conclusion

This document has presented the complete design and implementation of a capable embedded sensor node, tracing the evolution from a straightforward super-loop polling architecture to a production-grade FreeRTOS multi-task system with hardware-level deep sleep.

The baseline system established the key engineering decisions: binary framing with XOR integrity, hierarchical three-tier architecture (STM32 node → ESP-01 bridge → ESP32-S3 gateway), and compensation-driven gas concentration algorithms. These foundations are preserved in their entirety through the RTOS transition.

The Phase 2 enhancement replaces the polling-delay idle period with STM32 Stop mode, reducing average system current consumption from approximately 193 mA to 28 mA—a reduction of approximately 85 %. This is achieved through three coordinated mechanisms: power-switched sensor

rails managed by the PowerManagerTask, RTC-alarm-driven wake-up, and correct RTOS scheduler suspension around the Stop mode entry sequence. The R0 persistence strategy via RTC backup registers eliminates the need for repeated calibration and is robust across both soft resets and Stop mode cycles.

The platform as described is a solid foundation for a deployable, low-power environmental monitoring node. The architecture scales naturally to multi-node deployments via ESP-NOW peer registration and is extensible to additional sensing modalities without restructuring the task model.

References

1. STMicroelectronics. STM32F103xC/D/E Reference Manual (RM0008). Rev 21, 2023.
2. STMicroelectronics. STM32F103C8T6 Datasheet (DS5319). Rev 15, 2023.
3. FreeRTOS. FreeRTOS Kernel V10.3.1 Reference Manual. Amazon Web Services, 2021.
4. Hanwei Electronics. MQ-135 Air Quality Sensor Datasheet. 2022.
5. Aosong Electronics. DHT22 (AM2302) Sensor Specification Sheet. 2021.
6. Espressif Systems. ESP-NOW API Reference. ESP-IDF Programming Guide v5.x, 2024.
7. Solomon Systech. SSD1306 OLED/PLED Driver Datasheet Rev 1.1. 2020.
8. MathWorks. ThingSpeak IoT Analytics Documentation. 2024.

APPENDIX: Complete Task Implementations

A.1 MQ135Task — Full Implementation

```
void MQ135Task(void const * argument)
{
    for (;;) {
        osSemaphoreWait(sem_mql35_start, osWaitForever);

        /* Restore or calibrate R0 using RTC backup register sentinel */
        if (HAL_RTCEX_BKUPRead(&hrtc, R0_BKP_DR_MGC) == R0_BKP_MAGIC) {
            uint32_t r0_xl0 = HAL_RTCEX_BKUPRead(&hrtc, R0_BKP_DR_VAL);
            MQ135_SetR0((float)r0_xl0 / 10.0f);
        } else {
            float r0 = MQ135_CalibrateR0(MQ135_CAL_SAMPLES);
            HAL_RTCEX_BKUPWrite(&hrtc, R0_BKP_DR_VAL, (uint32_t)(r0 * 10.0f));
            HAL_RTCEX_BKUPWrite(&hrtc, R0_BKP_DR_MGC, R0_BKP_MAGIC);
        }

        /* Read temperature compensation values under mutex */
        float t_comp, h_comp;
        osMutexWait(mtx_sensor_data, osWaitForever);
        t_comp = dht_data.temperature;
        h_comp = dht_data.humidity;
        osMutexRelease(mtx_sensor_data);
        if (t_comp == 0.0f && h_comp == 0.0f) { t_comp=25.0f; h_comp=50.0f; }

        MQ135_GasReadings_t local_gas;
        if (MQ135_ReadAllGases(&local_gas, t_comp, h_comp)) {
            osMutexWait(mtx_sensor_data, osWaitForever);
            gas_data = local_gas;
            osMutexRelease(mtx_sensor_data);
        }
        osSemaphoreRelease(sem_mql35_done);
    }
}
```

A.2 DHT22Task — Full Implementation

```
void DHT22Task(void const * argument)
{
    for (;;) {
        osSemaphoreWait(sem_3v3_start, osWaitForever);
        osDelay(2000); // DHT22 needs 2s to stabilise after power-on

        DHT22_Data_t local_dht;
        if (DHT22_ReadData(&local_dht)) {
            osMutexWait(mtx_sensor_data, osWaitForever);
            dht_data = local_dht;
            osMutexRelease(mtx_sensor_data);
        }
        osSemaphoreRelease(sem_dht_done);
    }
}
```

A.3 Stop Mode Entry / Exit — PowerManagerTask (final steps)

```
/* Arm RTC alarm T+5 min, then enter Stop mode */
RTC_TimeTypeDef sTime = {0}; RTC_DateTypeDef sDate = {0};
HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);

RTC_AlarmTypeDef sAlarm = {0};
sAlarm.AlarmTime.Hours = sTime.Hours;
sAlarm.AlarmTime.Minutes = sTime.Minutes + 5;
sAlarm.AlarmTime.Seconds = sTime.Seconds;
if (sAlarm.AlarmTime.Minutes >= 60) {
    sAlarm.AlarmTime.Minutes -= 60;
    sAlarm.AlarmTime.Hours = (sAlarm.AlarmTime.Hours + 1) % 24;
}
sAlarm.Alarm = RTC_ALARM_A;
HAL_RTC_SetAlarm_IT(&hrtc, &sAlarm, RTC_FORMAT_BIN);

vTaskSuspendAll();
HAL_SuspendTick();
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
/* --- wake --- */
SystemClock_Config();
HAL_ResumeTick();
xTaskResumeAll();
```